

LAPORAN PRAKTIKUM DAN TUGAS

STRUKTUR DATA

“ADT”

DISUSUN OLEH:

Aliyatar rafi ahmad

2511533031

DOSEN PENGAMPU:

Dr. Wahyudi, S.T, M.T

ASISTEN PRAKTIKUM:

Sherly Sukmadira



DEPARTEMEN INFORMATIKA

FAKULTAS TEKNOLOGI INFORMASI

UNIVERSITAS ANDALAS

2025

KATA PENGANTAR

Puji syukur saya panjatkan kepada Tuhan Yang Maha Esa atas rahmat dan karunia-Nya sehingga laporan praktikum Struktur Data Pekan 1 ini dapat diselesaikan dengan baik.

Laporan ini disusun sebagai salah satu syarat untuk memenuhi tugas praktikum Struktur Data. Pada praktikum ini, saya mempelajari konsep Abstract Data Type (ADT) serta implementasinya dalam bahasa pemrograman Java.

Saya menyadari bahwa laporan ini masih memiliki kekurangan. Oleh karena itu, saya mengharapkan kritik dan saran yang membangun demi perbaikan di masa mendatang.

Saya juga mengucapkan terima kasih kepada asisten praktikum yang telah membimbing selama kegiatan praktikum berlangsung.

Padang, 01 April 2026

Aliyatar Rafi Ahmad

DAFTAR ISI

KATA PENGANTAR	i
DAFTAR ISI	ii
BAB I PENDAHULUAN	1
1.1 Latar Belakang	1
1.2 Tujuan Praktikum	1
1.3 Manfaat Praktikum	1
BAB II PEMBAHASAN	2
2.1 Pengertian Abstract Data Type (ADT)	2
2.2 Komponen Model ADT	2
2.3 Kode Program Java	3
2.3.1 Jam	4
2.3.2 Jam Driver	6
2.3.3 Jam Driver2	8
BAB III PENUTUP	11
3.1 Kesimpulan	11
DAFTAR PUSTAKA	12
LAPORAN TUGAS	13

BAB I

PENDAHULUAN

1.1 Latar Belakang

Struktur Data merupakan salah satu aspek penting dalam pemrograman yang digunakan untuk mengelola data secara efektif dan efisien. Salah satu konsep dasar dalam Struktur Data adalah Abstract Data Type (ADT), yaitu tipe data yang didefinisikan berdasarkan operasi yang dapat dilakukan tanpa memperhatikan detail implementasinya.

Pemahaman tentang ADT sangat penting karena menjadi dasar dalam mempelajari berbagai struktur data lainnya seperti ArrayList, Stack, Queue, dan Linked List. Dengan memahami ADT, mahasiswa dapat membuat program yang lebih terstruktur, sistematis, dan mudah dikembangkan.

Pada praktikum pekan pertama ini, mahasiswa diperkenalkan dengan konsep ADT serta implementasinya menggunakan bahasa Java untuk melatih kemampuan pemrograman.

1.2 Tujuan Praktikum

1. Memahami konsep Abstract Data Type (ADT).
2. Mampu mengimplementasikan ADT dalam bahasa pemrograman Java.
3. Melatih kemampuan dalam membuat program yang terstruktur.

1.3 Manfaat Praktikum

1. Menambah pemahaman mengenai konsep Abstract Data Type (ADT).
2. Melatih kemampuan dalam mengimplementasikan konsep ADT ke dalam bahasa pemrograman Java.
3. Menjadi dasar dalam mempelajari struktur data lainnya seperti Stack, Queue, dan Linked List.

BAB II

PEMBAHASAN

2.1 Pengertian Abstract Data Type (ADT)

Tipe Data Abstrak (Abstract Data Type/ADT) adalah konsep pemrograman yang mendefinisikan pandangan tingkat tinggi dari suatu struktur data, tanpa menentukan detail implementasinya. Dengan kata lain, ini adalah cetak biru untuk membuat struktur data yang mendefinisikan perilaku dan antarmuka struktur tersebut, tanpa menentukan bagaimana implementasinya.

ADT (Abstract Data Type) sering digunakan untuk mengabstraksi kompleksitas struktur data dan menyediakan antarmuka yang sederhana dan intuitif untuk mengakses dan memanipulasi data. Hal ini memudahkan programmer untuk memahami struktur data, dan menggunakannya dengan benar dalam program mereka.

2.2 Komponen Model ADT

Abstract Data Type (ADT) memiliki beberapa komponen utama yang membentuk modelnya. Komponen-komponen ini menjelaskan bagaimana data disimpan, digunakan, dan diakses tanpa memperlihatkan detail implementasi.

1. Data (Atribut)

Data merupakan nilai atau elemen yang disimpan dan dikelola dalam ADT. Komponen ini menjelaskan jenis dan struktur data yang digunakan, seperti bilangan bulat pada stack atau pasangan key-value pada map.

Contoh: integer pada stack, karakter pada queue.

2. Operasi (Method)

Operasi adalah fungsi atau metode yang digunakan untuk mengakses, menambah, menghapus, atau memanipulasi data dalam

ADT. Operasi ini hanya menjelaskan apa yang dilakukan, tanpa menjelaskan bagaimana cara kerjanya.

Contoh: insert(), delete(), search(), push(), pop(), enqueue().

3. Aturan atau Perilaku (Behavior)

Aturan atau perilaku menjelaskan bagaimana ADT bekerja serta batasan logis yang harus diikuti. Setiap ADT memiliki aturan tertentu agar hasil yang diperoleh sesuai dan konsisten.

4. Interface

Interface adalah kumpulan operasi yang dapat diakses oleh pengguna untuk berinteraksi dengan ADT. Interface hanya menampilkan fungsi yang tersedia tanpa menunjukkan proses internalnya, sehingga pengguna cukup memahami cara penggunaan tanpa mengetahui detail implementasi.

5. Implementasi

Implementasi merupakan bagian yang berisi kode atau struktur data yang digunakan untuk merealisasikan ADT, seperti array, linked list, atau struktur lainnya. Bagian ini bersifat tersembunyi dari pengguna dan memungkinkan fleksibilitas dalam pengembangan serta pemeliharaan program.

2.3 Kode Program Java

Pada praktikum pekan 1, Mahasiswa diinstruksikan untuk menyalin kode program mengenai ADT. Berikut program yang dibuat pada praktikum pekan 1.

2.3.1 Jam

```

1 package pekan1_2511533031;
2
3 public final class Jam_2511533031 {
4     private int hh; // 0..23
5     private int mm; // 0..59
6     private int ss; // 0..59
7
8     // --- validator ---
9     public static boolean isValid(int h, int m, int s) {
10         return (0 <= h && h <= 23) && (0 <= m && m <= 59) && (0 <= s && s <= 59);
11     }
12
13     // --- Konstruktor/Kreator ---
14     public Jam_2511533031(int h, int m, int s) {
15         this.hh = h; this.mm = m; this.ss = s;
16     }
17
18     // --- Selector ---
19     public int getHour() { return hh; }
20     public int getMinute() { return mm; }
21     public int getSecond() { return ss; }
22
23     // --- Mutator ---
24     public void setHour(int h) { this.hh = h; }
25     public void setMinute(int m) { this.mm = m; }
26     public void setSecond(int s) { this.ss = s; }
27
28     // --- Konversi ---
29     public int toSeconds() { return hh * 3600 + mm * 60 + ss; }
30     public static Jam_2511533031 fromSeconds(int total) {
31         if (total < 0) throw new IllegalArgumentException("detik negatif");
32         total %= 24 * 3600; // wrap 24 jam
33         int h = total / 3600; total %= 3600;
34         int m = total / 60; int s = total % 60;
35         return new Jam_2511533031(h, m, s);
36     }
37
38     // --- Relasional ---
39     public int compareTo(Jam_2511533031 other) { return Integer.compare(this.toSeconds(), other.toSeconds()); }
40     public boolean equals(Object o) {
41         if (!(o instanceof Jam_2511533031 j)) return false;
42         return hh == j.hh && mm == j.mm && ss == j.ss;
43     }
44     public int hashCode() { return java.util.Objects.hash(hh, mm, ss); }
45
46     // --- Aritmatika ---
47     public Jam_2511533031 plus(Jam_2511533031 other) { return fromSeconds(this.toSeconds() + other.toSeconds()); }
48     public Jam_2511533031 minus(Jam_2511533031 other) { return fromSeconds(Math.floorMod(this.toSeconds() - other.toSeconds(), 24 * 3600)); }
49     public Jam_2511533031 nextSecond() { return fromSeconds(this.toSeconds() + 1); }
50     public Jam_2511533031 nextNSeconds(int n) { return fromSeconds(this.toSeconds() + Math.max(0, n)); }
51     public Jam_2511533031 prevSecond() { return fromSeconds(Math.floorMod(this.toSeconds() - 1, 24 * 3600)); }
52     public Jam_2511533031 prevNSeconds(int n) { return fromSeconds(Math.floorMod(this.toSeconds() - Math.max(0, n), 24 * 3600)); }
53
54     // --- Durasi (detik) --- bisa negatif jika this > other? sesuai spesifikasi durasi (Jaw, Jakh)
55     public static int durasiDetik(Jam_2511533031 jam, Jam_2511533031 jak) { return jak.toSeconds() - jam.toSeconds(); }
56     // overriding ke string
57     public String toString() { return String.format("%02d:%02d:%02d", hh, mm, ss); }
58 }

```

Gambar 2.1

Penjelasan Program:

Program di atas merupakan implementasi Abstract Data Type (ADT) **Jam** menggunakan bahasa pemrograman Java. ADT ini digunakan untuk merepresentasikan waktu dalam satuan jam, menit, dan detik, serta menyediakan berbagai operasi yang dapat dilakukan terhadap data waktu tersebut.

• 1. Atribut (Data)

Pada class `Jam_2511533031` terdapat tiga atribut utama, yaitu:

- `hh`: menyimpan nilai jam (0–23)
- `mm`: menyimpan nilai menit (0–59)
- `ss`: menyimpan nilai detik (0–59)

Ketiga atribut ini bersifat `private`, sehingga tidak dapat diakses langsung dari luar class (sesuai konsep enkapsulasi pada ADT).

- **2. Validator**

Method `isValid(int h, int m, int s)` digunakan untuk mengecek apakah nilai jam, menit, dan detik valid atau tidak. Jika nilai berada dalam rentang yang sesuai, maka method akan mengembalikan nilai `true`.

- **3. Konstruktor**

Konstruktor `Jam_2511533031(int h, int m, int s)` digunakan untuk membuat objek jam dengan nilai awal tertentu.

- **4. Selector (Getter)**

Method `getter` digunakan untuk mengambil nilai atribut:

- `getHour()` → mengambil jam
- `getMinute()` → mengambil menit
- `getSecond()` → mengambil detik

- **5. Mutator (Setter)**

Method `setter` digunakan untuk mengubah nilai atribut:

- `setHour()`
- `setMinute()`
- `setSecond()`

- **6. Konversi**

- `toSeconds()` → mengubah waktu menjadi total detik
- `fromSeconds()` → mengubah total detik menjadi format jam, menit, detik

Method ini memudahkan perhitungan waktu secara matematis.

- **7. Operasi Relasional**

- `compareTo()` → membandingkan dua objek jam
- `equals()` → mengecek apakah dua waktu sama
- `hashCode()` → digunakan untuk keperluan hashing

- **8. Operasi Aritmatika Waktu**

Class ini menyediakan operasi perhitungan waktu, seperti:

- `plus()` → penjumlahan dua waktu
- `minus()` → pengurangan waktu
- `nextSecond()` → menambah 1 detik

- `prevSecond()` → mengurangi 1 detik

Operasi ini menggunakan konsep konversi ke detik agar lebih mudah dihitung.

• 9. Durasi Waktu

Method `durasiDetik()` digunakan untuk menghitung selisih waktu dalam satuan detik antara dua objek jam.

• 10. Output (`toString`)

Method `toString()` digunakan untuk menampilkan waktu dalam format **HH:MM:SS**, sehingga mudah dibaca oleh pengguna.

• Kesimpulan Pembahasan

Program ini telah mengimplementasikan konsep ADT dengan baik, dimana data disembunyikan (enkapsulasi) dan hanya dapat diakses melalui method yang tersedia. Selain itu, program juga menyediakan berbagai operasi yang lengkap untuk memanipulasi data waktu.

2.3.2 Jam Driver

```

1 package pekan1_2511533031;
2
3 public class JamDriver_2511533031 {
4
5     public static void main(String[] args) {
6         Jam_2511533031 a = new Jam_2511533031(23, 59, 50);
7         Jam_2511533031 b = new Jam_2511533031(0, 0, 15);
8         System.out.println("a      = " + a);
9         System.out.println("b      = " + b);
10        System.out.println("a+b    = " + a.plus(b));
11        System.out.println("next 20s = " + a.nextNSeconds(20));
12        System.out.println("durasi (a,b) = " + Jam_2511533031.durasiDetik(a, b));
13        System.out.println("a.compareTo (b) = " + a.compareTo(b));
14    }
15 }
16 }

```

Gambar 2.2

Class `JamDriver_2511533031` merupakan program utama yang digunakan untuk menguji dan menjalankan fungsi-fungsi yang terdapat pada class `Jam_2511533031`.

Pada method `main`, dibuat dua objek jam, yaitu:

- a dengan nilai **23:59:50**
- b dengan nilai **00:00:15**

Selanjutnya dilakukan beberapa operasi sebagai berikut:

- `System.out.println("a = " + a);`
Menampilkan nilai waktu objek a.
- `System.out.println("b = " + b);`
Menampilkan nilai waktu objek b.
- `a.plus(b)`
Menjumlahkan dua waktu. Karena menggunakan sistem 24 jam, hasil akan menyesuaikan (wrap ke hari berikutnya jika lewat 24 jam).
- `a.nextNSeconds(20)`
Menambahkan 20 detik ke waktu a. Dari 23:59:50 akan menjadi 00:00:10 (melewati batas hari).
- `Jam_2511533031.durasiDetik(a, b)`
Menghitung selisih waktu dalam detik antara a dan b.
- `a.compareTo(b)`
Membandingkan dua waktu. Jika hasil negatif berarti $a < b$, jika positif berarti $a > b$, dan jika nol berarti sama.

Output Program

```
<terminated> JamDriver_2511533031 [Java Ap
a          = 23:59:50
b          = 00:00:15
a+b        = 00:00:05
next 20s = 00:00:10
durasi (a,b) = -86375
a.compareTo (b) = 1
```

Gambar 2.3

Penjelasan Output

- **a = 23:59:50**

Menampilkan objek a dengan waktu 23 jam, 59 menit, dan 50 detik.

- **b = 00:00:15**

Menampilkan objek b dengan waktu 0 jam, 0 menit, dan 15 detik.

- **a + b = 00:00:05**

Hasil penjumlahan waktu a dan b. Total detik melebihi 24 jam sehingga kembali ke awal hari (00:00:05).

- **next 20s = 00:00:10**

Menambahkan 20 detik pada waktu a. Karena melewati batas 24 jam, hasilnya menjadi 00:00:10.

- **durasi (a, b) = -86375**

Menunjukkan selisih waktu dalam detik antara a dan b. Nilai negatif berarti waktu a lebih besar dari b.

- **a.compareTo(b) = 1**

Menunjukkan bahwa waktu a lebih besar dari b, karena hasil perbandingan bernilai positif.

2.3.3 Jam Driver2

```

1 package pekan1_2511533031;
2 import java.util.Scanner;
3 public class JamDriver2_2511533031 {
4     public static void main(String[] args) {
5         Scanner input = new Scanner(System.in);
6         System.out.println("=== Program Driver objek Jam ===");
7         // 1. Input jam pertama
8         System.out.println("n--- Input Jam 1 ---");
9         Jam_2511533031 j1 = buatJamDariInput(input);
10
11         // 2. Input jam kedua
12         System.out.println("n--- Input Jam 2 ---");
13         Jam_2511533031 j2 = buatJamDariInput(input);
14
15         // 3. Menampilkan data
16         System.out.println("n--- Hasil Operasi ---");
17         System.out.println("Jam 1 (String) : " + j1.toString());
18         System.out.println("Jam 2 (String) : " + j2.toString());
19         System.out.println("Jam 1 dalam detik : " + j1.toSeconds());
20         System.out.println("Jam 2 dalam detik : " + j2.toSeconds());
21
22         // 4. Operasi Relasional (Perbandingan)
23         int perbandingan = j1.compareTo(j2);
24         if (perbandingan > 0) {
25             System.out.println("Status : Jam 1 lebih lambat (setelah) Jam 2");
26         } else if (perbandingan < 0) {
27             System.out.println("Status : Jam 1 lebih awal (sebelum) Jam 2");
28         } else {
29             System.out.println("Status : Jam 1 dan Jam 2 sama persis");
30         }
31
32         // 5. Operasi Aritmatika
33         System.out.println("Durasi (J1 ke J2) : " + Jam_2511533031.durasiDetik(j1, j2) + " detik ");
34
35         Jam_2511533031 jnext = j1.nextSecond();
36         System.out.println("Jam 1 detik berikutnya: " + jnext);
37
38         Jam_2511533031 jprev = j1.prevSecond();
39         System.out.println("Jam 1 detik sebelumnya: " + jprev);
40
41         // 6. Operasi penjumlahan jam
42         Jam_2511533031 jHasilPlus = j1.plus(j2);
43         System.out.println("Hasil J1 + J2 : " + jHasilPlus);
44
45         input.close();
46         System.out.println("nProgram Selesai.");
47     }
48
49     private static Jam_2511533031 buatJamDariInput(Scanner sc) {
50         int h, m, s;
51         while (true) {
52             System.out.print("Masukkan Jam (0-23) :");
53             h = sc.nextInt();
54             System.out.print("Masukkan Menit (0-59) :");
55             m = sc.nextInt();
56             System.out.print("Masukkan detik (0-59) :");
57             s = sc.nextInt();
58
59             // Memanggil method static isValid dari kelas Jam
60             if (Jam_2511533031.isValid(h, m, s)) {
61                 return new Jam_2511533031 (h, m, s);
62             } else {
63                 System.out.println("[Error] input tidak valid! Silahkan ulangi");
64             }
65         }
66     }
67 }
68
69 }
70

```

Gambar 2.4

Class Jamdriver2 merupakan program utama yang digunakan untuk menguji ADT Jam dengan input dari pengguna. Program ini memungkinkan pengguna memasukkan dua waktu, kemudian dilakukan berbagai operasi terhadap kedua objek tersebut.

Program menggunakan Scanner untuk menerima input jam, menit, dan detik dari user. Input akan divalidasi menggunakan method isValid() agar sesuai dengan format waktu yang benar.

Alur Program

1. Program meminta input **Jam 1** dan **Jam 2** dari pengguna
2. Menampilkan data kedua jam dalam bentuk string dan detik
3. Melakukan perbandingan antara dua waktu
4. Menghitung durasi waktu
5. Menampilkan operasi aritmatika seperti next second, previous second, dan penjumlahan waktu

Output

```

terminated> Jamdriver2_2511533031 [Java Application] C:\Users\User\p2\pool\plugins\c
=== Program Driver objek Jam ===

--- Input Jam 1 ---
Masukkan Jam (0-23) :12
Masukkan Menit (0-59) :45
Masukkan detik (0-59) :56

--- Input Jam 2 ---
Masukkan Jam (0-23) :21
Masukkan Menit (0-59) :43
Masukkan detik (0-59) :3

--- Hasil Operasi ---
Jam 1 (String) : 12:45:56
Jam 2 (String) : 21:43:03
Jam 1 dalam detik : 45956
Jam 2 dalam detik : 78183
Status : Jam 1 lebih awal (sebelum) Jam 2
Durasi (J1 ke J2) : 32227 detik
Jam 1 detik berikutnya: 12:45:57
Jam 1 detik sebelumnya: 12:45:55
Hasil J1 + J2 : 10:28:59

Program Selesai.

```

Gambar 2.5

Penjelasan Output

• Jam 1 (String)

Menampilkan waktu pertama dalam format HH:MM:SS.

• Jam 2 (String)

Menampilkan waktu kedua dalam format HH:MM:SS.

- **Jam 1 dalam detik**

Mengubah waktu Jam 1 menjadi total detik.

- **Jam 2 dalam detik**

Mengubah waktu Jam 2 menjadi total detik.

- **Operasi Relasional**

- Jika $j1 > j2$

→ ditampilkan “Jam 1 lebih lambat (setelah) Jam 2”

- Jika $j1 < j2$

→ ditampilkan “Jam 1 lebih awal (sebelum) Jam 2”

- Jika sama

→ ditampilkan “Jam 1 dan Jam 2 sama persis”

- **Operasi Aritmatika**

- **Durasi (J1 ke J2)**

Menghitung selisih waktu dalam detik antara Jam 1 dan Jam 2.

- **Jam 1 detik berikutnya**

Menampilkan waktu setelah ditambah 1 detik.

- **Jam 1 detik sebelumnya**

Menampilkan waktu setelah dikurangi 1 detik.

- **Hasil J1 + J2**

Menampilkan hasil penjumlahan dua waktu dengan sistem 24 jam (akan kembali ke 00 jika melewati 24 jam).

- **Validasi Input**

Method `buatJamDariInput()` digunakan untuk memastikan bahwa input yang dimasukkan pengguna valid. Jika input tidak sesuai (misalnya $jam > 23$), maka program akan meminta pengguna mengulang input hingga benar.

BAB III

PENUTUP

3.1 Kesimpulan

Berdasarkan praktikum yang telah dilakukan, dapat disimpulkan bahwa Abstract Data Type (ADT) merupakan konsep penting dalam struktur data yang berfokus pada operasi terhadap data tanpa memperhatikan detail implementasinya.

Pada praktikum ini, ADT Jam berhasil diimplementasikan menggunakan bahasa pemrograman Java dengan memanfaatkan konsep class dan object. Program yang dibuat mampu melakukan berbagai operasi seperti konversi waktu, perbandingan, penjumlahan, serta manipulasi waktu lainnya.

Selain itu, penggunaan driver program membantu dalam menguji fungsi-fungsi yang telah dibuat, baik secara langsung maupun melalui input dari pengguna.

DAFTAR PUSTAKA

- [1] PREPBYTES, "Abstract Data Type (ADT) in Data Structure," PrepBytes Blog, 24 March 2023. [Online]. Available: <https://prepbytes.com/blog/abstract-data-type-adt-in-data-structure/>. [Accessed 31 March 2026].

- [2] A. Verma, "Abstract Data Type in Data Structure," Intellipaat, 29 July 2025. [Online]. Available: <https://intellipaat.com/blog/abstract-data-type/>. [Accessed 31 March 2026].

LAPORAN TUGAS

A. Kode Program

1. Class Mobil

```
1 package pekan1_2511533031;
2
3 public class Mobil_2511533031 {
4
5     private String nama;
6     private int tahun;
7     private int cc;
8     private double harga;
9     private String merk;
10
11     // constructor
12     public Mobil_2511533031(String nama, int tahun, int cc, double harga, String merk) {
13         this.nama = nama;
14         this.tahun = tahun;
15         this.cc = cc;
16         this.harga = harga;
17         this.merk = merk;
18     }
19
20     // ===== SELECTOR (getter)
21     public String getNama() { return nama; }
22     public int getTahun() { return tahun; }
23     public int getCc() { return cc; }
24     public double getHarga() { return harga; }
25     public String getMerk() { return merk; }
26
27     // ===== MUTATOR (setter)
28     public void setHarga(double harga) {
29         this.harga = harga;
30     }
31
32     // ===== TAMBAH MOBIL (membuat objek baru)
33     public static Mobil_2511533031 tambahMobil(String nama, int tahun, int cc, double harga, String merk) {
34         return new Mobil_2511533031(nama, tahun, cc, harga, merk);
35     }
36
37     // ===== HAPUS MOBIL (mengosongkan data)
38     public void hapusMobil() {
39         this.nama = "";
40         this.tahun = 0;
41         this.cc = 0;
42         this.harga = 0;
43         this.merk = "";
44     }
45
46     // tampilkan data
47     public void tampilkan() {
48         System.out.println("Nama : " + nama);
49         System.out.println("Tahun : " + tahun);
50         System.out.println("CC : " + cc);
51         System.out.println("Harga : " + harga);
52         System.out.println("Merk : " + merk);
53     }
54 }
```

Class Mobil_2511533031 berfungsi sebagai class utama yang merepresentasikan objek mobil. Di dalam class ini terdapat beberapa atribut yaitu nama, tahun, kapasitas mesin (cc), harga, dan merk. Class ini juga memiliki constructor yang digunakan untuk menginisialisasi nilai awal objek saat dibuat.

Selain itu, terdapat method getter yang digunakan untuk mengambil nilai atribut dan setter yang digunakan untuk mengubah nilai atribut, seperti perubahan harga mobil. Method tambahMobil() digunakan untuk membuat objek mobil baru, sedangkan method hapusMobil() digunakan untuk mengosongkan data mobil. Method tampilkan() berfungsi untuk menampilkan data mobil ke layar.

B. Class Driver

```

3 public class Driver_2511533031 {
4     public static void main(String[] args) {
5
6         // tambah mobil
7         Mobil_2511533031 mobil1 = Mobil_2511533031.tambahMobil(
8             "Fortuner", 2022, 2400, 550000000, "Toyota"
9         );
10
11        Mobil_2511533031 mobil2 = Mobil_2511533031.tambahMobil(
12            "Pajero", 2021, 2500, 600000000, "Mitsubishi"
13        );
14
15        // tampil awal
16        System.out.println("=== DATA AWAL ===");
17        System.out.println("\nMobil 1:");
18        mobil1.tampilkan();
19
20        System.out.println("\nMobil 2:");
21        mobil2.tampilkan();
22
23        // update harga
24        mobil1.setHarga(530000000);
25        mobil2.setHarga(550000000);
26
27        System.out.println("\n=== SETELAH UPDATE HARGA ===");
28        System.out.println("\nMobil 1:");
29        mobil1.tampilkan();
30
31        System.out.println("\nMobil 2:");
32        mobil2.tampilkan();
33
34        // hapus mobil1
35        mobil1.hapusMobil();
36
37        System.out.println("\n=== SETELAH HAPUS MOBIL 1 ===");
38        System.out.println("\nMobil 1:");
39        mobil1.tampilkan();
40
41        System.out.println("\nMobil 2:");
42        mobil2.tampilkan();
43    }
44 }

```

Class `Driver_2511533031` merupakan class yang berisi method `main` sebagai program utama. Pada class ini dilakukan pembuatan dua objek mobil, yaitu `mobil1` dan `mobil2`. Selanjutnya, program menampilkan data awal mobil, kemudian melakukan perubahan harga menggunakan method `setter`.

Setelah itu, program kembali menampilkan data yang telah diperbarui. Kemudian salah satu objek mobil dihapus menggunakan method `hapusMobil()`, dan hasil akhirnya ditampilkan kembali ke layar.

Secara keseluruhan, program ini berhasil mengimplementasikan konsep dasar OOP seperti class, object, constructor, method, serta proses manipulasi data seperti menambah, mengubah, dan menghapus data.